

Name : Vaishnavi Janardhan Tapase PRN : 202501030031 Batch : CV2

IMDB Ratings Data Analysis

This notebook contains an analysis of IMDB ratings, exploring central tendencies, distributions, and correlations.

```
import pandas as pd
df = pd.read_csv('/content/imdb_ratings.csv')
df
```

Unnamed: 0	titleId	title	averageRating	numVotes	
0	0	tt0000001	Carmencita	5.7	1676
1	1	tt0000002	Le clown et ses chiens	6.1	208
2	2	tt0000003	Pauvre Pierrot	6.5	1404
3	3	tt0000004	Un bon bock	6.2	123
4	4	tt0000005	Blacksmith Scene	6.2	2200
...	...	...	...	...	...
67403	67404	tt9916270	Il talento del calabrone	5.7	853
67404	67405	tt9916362	Akelarre	5.8	199
67405	67406	tt9916428	Hong xing zhao yao Zhong guo	3.8	12
67406	67407	tt9916460	Pink Taxi	9.2	12
67407	67408	tt9916720	The Nun 2	6.2	72

67408 rows × 5 columns

Next steps: [Generate code with df](#) [New interactive sheet](#)

1. Average Rating

Calculating the mean of all movie ratings in the dataset.

```
df['averageRating'].mean()
np.float64(6.354567707097082)
```

2. Highest Rated Title

Finding the entry with the maximum `averageRating`.

```
df.loc[df['averageRating'].idxmax()]
```

	30328
Unnamed: 0	30328
titleId	tt10407660
title	Kim Hushable
averageRating	10.0
numVotes	6

dtype: object

3. Lowest Rated Title

Finding the entry with the minimum `averageRating`.

```
df.loc[df['averageRating'].idxmin()]
```

1783	
Unnamed: 0	1783
titleId	tt0010319
title	The Jungle Trail
averageRating	1.0
numVotes	8
dtype: object	

4. Dataset Size

Counting the total number of records.

```
len(df)
```

67408

5. High-Rated Movies

Filtering movies with an average rating greater than 8.

```
df[df['averageRating'] > 8]
```

Unnamed: 0		titleId	title	averageRating	numVotes	
195	195	tt0000417	Le voyage dans la lune	8.2	44126	
227	227	tt0000505	La course à la perruque	8.1	11	
470	470	tt0001456	White Roses	8.5	8	
574	574	tt0002145	The Dream of a Moving Picture Director	8.7	7	
593	593	tt0002234	Ett hemligt giftermål	8.5	13	
...	...	...	...	...	...	
67365	67366	tt9909248	Race Across the World	8.3	404	
67388	67389	tt9914642	Albatross	8.2	28	
67389	67390	tt9914644	9/11: Escape from the Towers	8.3	65	
67392	67393	tt9915144	Never Ever	9.3	47	
67406	67407	tt9916460	Pink Taxi	9.2	12	

5971 rows × 5 columns

6. Popular Movies

Filtering movies with more than 10,000 votes.

```
df[df['numVotes'] > 10000]
```

	Unnamed: 0	titleId	title	averageRating	numVotes	
11	11	tt0000012	L'arrivée d'un train à La Ciotat	7.4	10623	
195	195	tt0000417	Le voyage dans la lune	8.2	44126	
203	203	tt0000439	The Great Train Robbery	7.3	17359	
931	931	tt0004972	The Birth of a Nation	6.3	22736	
1207	1207	tt0006864	Intolerance: Love's Struggle Throughout the Ages	7.7	14231	
...	...	...	...	...	...	
66882	66883	tt9827854	Hollywood	7.6	29194	
67079	67080	tt9860728	Falling Inn Love	5.6	15762	
67116	67117	tt9866072	Holidate	6.1	43892	
67309	67310	tt9898858	Coffee & Kareem	5.1	11638	
67329	67330	tt9900782	Kaithi	8.5	12117	

1962 rows × 5 columns

7. Average Vote Count

Calculating the mean number of votes across the dataset.

```
df['numVotes'].mean()
np.float64(2408.153631616425)
```

8. Top 10 by Popularity (Votes)

Sorting the dataset to find the ten most-voted titles.

```
df.sort_values(by='numVotes', ascending=False).head(10)
```

	Unnamed: 0	titleId	title	averageRating	numVotes	
44770	44771	tt1345836	The Dark Knight Rises	8.4	1515727	
35182	35182	tt1130884	Shutter Island	8.2	1129155	
18956	18956	tt0088763	Back to the Future	8.5	1057549	
17122	17122	tt0086190	Star Wars: Episode VI - Return of the Jedi	8.3	950081	
46801	46802	tt7286456	Joker	8.5	937904	
30889	30889	tt1049413	Up	8.2	935115	
45611	45612	tt1392190	Mad Max: Fury Road	8.1	881816	
45608	45609	tt1392170	The Hunger Games	7.2	844312	
18599	18599	tt0088247	The Terminator	8.0	799595	
28587	28587	tt1010048	Slumdog Millionaire	8.0	798653	

9. Top 10 by Rating

Sorting the dataset to find the ten highest-rated titles.

```
df.sort_values(by='averageRating', ascending=False).head(10)
```

	Unnamed: 0	titleId	title	averageRating	numVotes	
30328	30328	tt10407660	Kim Hushable	10.0	6	
38565	38565	tt1181817	God's Will: The Sex Factor	10.0	5	
38513	38513	tt11809482	Sadika: l'Artiste Multidimensionnelle	10.0	7	
34771	34771	tt11191356	Dear Future Children	10.0	25	
41631	41631	tt12545522	Cityscapes	10.0	17	
42151	42151	tt12728950	The Wake-Up Call with Pritan Ambroase	10.0	723	
44528	44529	tt13397588	What are we waiting for?	10.0	8	
36347	36347	tt11460966	Ines Pedretti contro i sensi vietati, le strad...	10.0	8	
44115	44116	tt13220946	Shubhmangal Online	10.0	5	
32748	32748	tt10773030	Aesop's Fables	10.0	7	

10. Correlation Analysis

Checking the correlation between rating and the number of votes.

```
df[['averageRating', 'numVotes']].corr()
```

	averageRating	numVotes
averageRating	1.00000	0.06343
numVotes	0.06343	1.00000

11. Mid-Range Ratings Count

Count of movies with ratings between 6 and 7 inclusive.

```
df[(df['averageRating'] >= 6) & (df['averageRating'] <= 7)].shape[0]
```

23434

12. Maximum Popularity Score

Identifying the title with the highest calculated popularity score.

```
df.loc[df['popularity'].idxmax()]
```

	44770
Unnamed: 0	44771
titleId	tt1345836
title	The Dark Knight Rises
averageRating	8.4
numVotes	1515727
popularity	12732106.8
dtype:	object

13. Rating Statistics (Median & Std Dev)

Calculating the median and standard deviation of the ratings.

```
df['averageRating'].median()
```

6.5

```
df['averageRating'].std()
```

1.3517184556833448

14. Above Average Performance

Counting how many movies have a rating higher than the overall mean.

```
avg = df['averageRating'].mean()
df[df['averageRating'] > avg].shape[0]
```

36574

15. Rating Categorization

Binning movies into 'Low', 'Medium', and 'High' categories based on their ratings.

```
bins = [0, 5, 7, 10]
labels = ['Low', 'Medium', 'High']
df['category'] = pd.cut(df['averageRating'], bins=bins, labels=labels)
df['category'].value_counts()
```

count	
category	
Medium	35892
High	21107
Low	10409

dtype: int64

16. The 'Hidden Gems' or 'Flops'

Finding movies with low ratings despite high vote counts.

```
df[(df['averageRating'] < 5) & (df['numVotes'] > 5000)]
```

Unnamed: 0		titleId	title	averageRating	numVotes	popularity	category
5378	5378	tt0047898	Bride of the Monster	4.2	6562	27560.4	Low
8556	8556	tt0052077	Plan 9 from Outer Space	4.0	35992	143968.0	Low
8988	8988	tt0052646	The Brain That Wouldn't Die	4.5	6246	28107.0	Low
10540	10540	tt0054673	The Beast of Yucca Flats	2.2	8019	17641.8	Low
11536	11536	tt0055946	Eegah	2.2	5413	11908.6	Low
...	...	...	...	...	...	...	...
65298	65299	tt9558612	PM Narendra Modi	3.0	5532	16596.0	Low
65691	65692	tt9625664	Trauma Center	4.4	6469	28463.6	Low
65837	65838	tt9648672	Street Dancer 3D	3.5	5552	19432.0	Low
66128	66129	tt9691136	Shadow in the Cloud	4.7	9513	44711.1	Low
66839	66840	tt9820556	Breach	3.7	5505	20368.5	Low

161 rows × 7 columns

17. Data Normalization

Normalizing ratings using Z-score calculation.

```
import numpy as np
df['normalized_rating'] = (df['averageRating'] - np.mean(df['averageRating'])) / np.std(df['averageRating'])
```

18. Vote Percentiles

Calculating the 25th, 50th, and 75th percentiles for vote counts.

```
np.percentile(df['numVotes'], [25, 50, 75])

array([ 18.,  75., 368.])
```

19. Highly Voted but Low Rated (Cult Classics or Flops?)

Finding movies with more than 50,000 votes but a rating below 5.0.

```
df[(df['numVotes'] > 50000) & (df['averageRating'] < 5)]
```

Unnamed: 0	titleId	title	averageRating	numVotes	
29866	29866 tt1029360	Poltergeist	4.9	54986	
30626	30626 tt1043726	The Legend of Hercules	4.2	52591	
30856	30856 tt1045778	Year One	4.9	92100	
32184	32184 tt1071875	Ghost Rider: Spirit of Vengeance	4.3	110963	
32333	32333 tt1073498	Meet the Spartans	2.8	101486	
32576	32576 tt1075747	Jonah Hex	4.7	55400	
33930	33930 tt1093357	The Darkest Hour	4.9	57846	
34050	34050 tt1098327	Dragonball Evolution	2.5	70258	
41717	41717 tt1259571	The Twilight Saga: New Moon	4.7	265745	
41902	41902 tt1261945	Sex and the City 2	4.5	74335	
43986	43987 tt1320261	Gulliver's Travels	4.9	65652	
50405	50406 tt7668842	Enes Batur Hayal mi Gerçek mi?	4.0	54116	
52388	52389 tt7886848	Sadak 2	1.1	64686	

20. Rating Frequency Distribution

Counting how many movies fall into each integer rating (1 to 10).

```
df['averageRating'].round().value_counts().sort_index()
```

	count
averageRating	
1.0	122
2.0	645
3.0	1369
4.0	4296
5.0	8589
6.0	20195
7.0	18742
8.0	11095
9.0	2181
10.0	174

dtype: int64

21. Top 5 Movies with Exactly 10.0 Rating and Most Votes

Finding the most 'credible' perfect-rated titles.

```
df[df['averageRating'] == 10.0].sort_values(by='numVotes', ascending=False).head(5)
```